TI-2400 Processamento de Linguagem Natural

ANTONIO CARVALHO - TREINAMENTOS

Pandas

Tradução adaptada da apresentação do *Dr. Ziad Al-Sharif*

O que é Pandas

Pandas é uma poderosa biblioteca para Python utilizada para trabalhar com conjuntos de dados.

Possui funções para analisar, limpar, e explorar e manipular dados.

Como instalar

Instalar o pandas é facil, se estiver utilizando uma instalação do python basta acessar o terminal do windows ou do linux e digitar:

- **pip install pandas**

ou

- **python –m pip install pandas**

Se estiver usando o anaconda, digite:

- **conda install pandas**

Se deseja instalar a partir do Notebook Jupyter execute:

- **!pip install pandas**
  - Usar o ! no início faz com que o comando seja executado como se estivesse no terminal

Documentação sobre a instalação pode ser acessada em:

https://pandas.pydata.org/pandas-docs/stable/getting_started/install.html

O que é Pandas

Os dados no Pandas são armazenados em dois formatos fundamentais:

- **Series** é o tipo de dado que armazena apenas uma simples coluna de dados. Similar ao um vetor no Numpy array, porém nomeada
- **DataFrame** é o tipo de dados que armazena um conjunto de dados com várias colunas. Similar a uma matriz do Numpy array, porém nomeadas

O que é Pandas

Series como consistem em uma coluna simples possuem apenas um índice

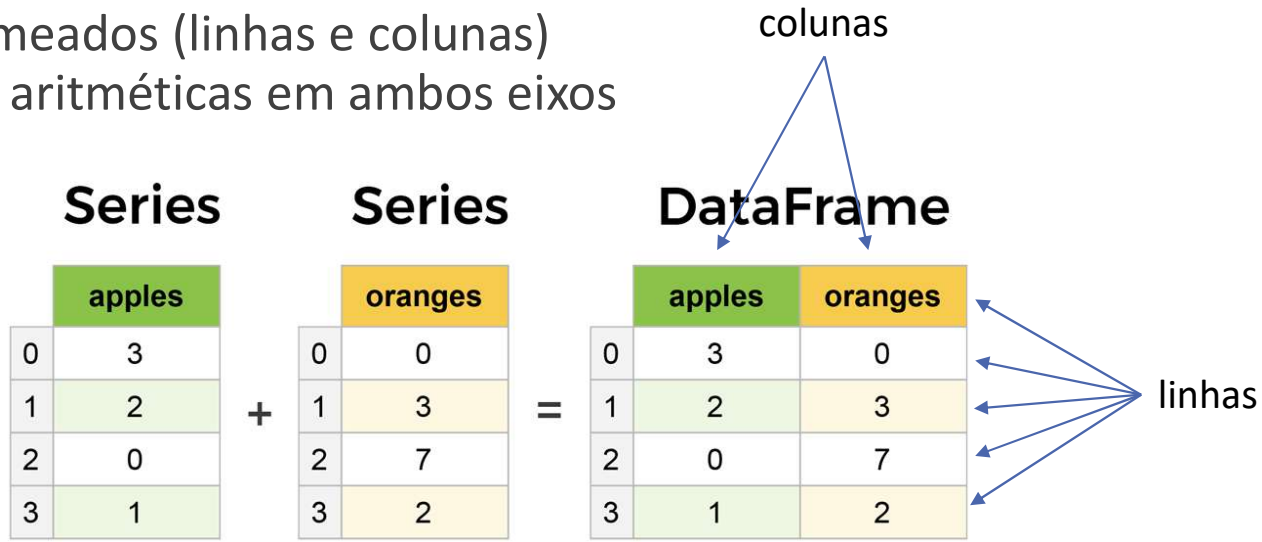
Series

	apples
0	3
1	2
2	0
3	1

O que é Pandas

Dataframe é uma estrutura bidimensional composta por um conjunto de **series**

- As colunas podem ser de tipos diferentes
- O tamanho pode ser modificado
- Ambos eixos podem ser nomeados (linhas e colunas)
- Podem ser feitas operações aritméticas em ambos eixos



Como criar um DataFrame

Sintaxe para criar um dataframe

```
import pandas as pd

pd.DataFrame( dados, índices, colunas, dtype, copy )
```

S

- **dados**, tipos de dados aceitos em vários formatos, **ndarray**, **series**, **map**, **listas**, **dict**, **constantes** e outro **Dataframe**
- **indices**, para nomear as linhas, é opcional e caso não seja informado será um **`np.arrange(n)`**
- **colunas**, para nomear as colunas, é opcional e caso não seja informado será um - **`np.arrange(n)`**
- **dtype**, tipo de dado para cada coluna
- **copy**, booleano indica se será criada uma cópia

Criar um Dataframe do zero

Uma das maneiras mais simples de se criar um Dataframe é a partir de um dicionario

```
import pandas as pd
```

Imagine que você possui um stande de frutas como maças e laranjas, e precisa organizer este stand de maneira que cada fileira tenha uma quantidade de frutas :

```
data = { 'apples':[3, 2, 0, 1] , 'oranges':[0, 3, 7, 2] }

df = pd.DataFrame(data)
```

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Como isto funciona

Cada item com (chave, valor) corresponde a uma coluna no DataFrame resultante.

O indice desde DataFrame inicialmente consiste em números de 0 a 3, porém é póssível nomear os indices de maneiras alternativas conforme mostrado a seguir.

Exemplo: Se deseja colocar os nomes dos clientes nos indices:

```
df = pd.DataFrame(data, index=['Ahmad', 'Ali', 'Rashed', 'Hamza'])
```

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

- Agora os dados podem ser acessados por meio do nome

```
df.loc['Ali']
```

```
apples      2
oranges     3
Name: Ali, dtype: int64
```

Criar DataFrame apatir de Dict

```
pandas.DataFrame.from_dict(data, orient='columns', dtype=None, columns=None)
```

- **data** : dict
  - No formato `{field:array-like}` ou `{field:dict}`.

- **orient** : `{'columns', 'index'}`, padrão '`columns`'
  - A orientação dos dados.
  - Se as chaves dos dicionários forem as colunas, passe 'columns' (default).
  - Caso contrário se as chaves forem ser as linhas, passe 'index'.

- **dtype** : `dtype`, default `None`
  - Data type para forçar os dados, se não for passado o pandas irá identificar o tipo dos dados existentes

- **columns** : `list`, default `None`
  - Labels das colunas utilizadas quando a orientação for `orient='index'`. Lança uma `ValueError` se usado quanto a orientação for `orient='columns'`.

https://pandas.pydata.org/pandas-docs/version/0.23/generated/pandas.DataFrame.from_dict.html

Orientação

```
data = {'col_1':[3, 2, 1, 0], 'col_2':['a','b','c','d']}
pd.DataFrame.from_dict(data)
```

	col_1	col_2
0	3	a
1	2	b
2	1	c
3	0	d

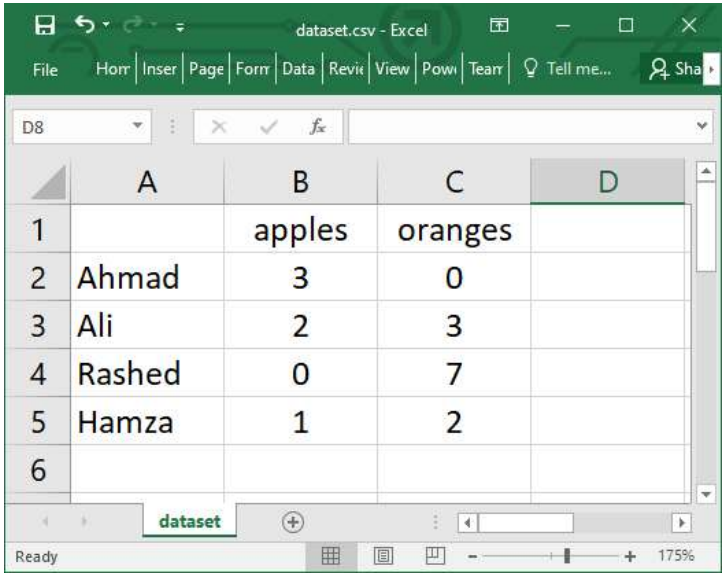
```
data = {'row_1':[3, 2, 1, 0], 'row_2':['a','b','c','d']}
pd.DataFrame.from_dict(data,
                       orient='index')
```

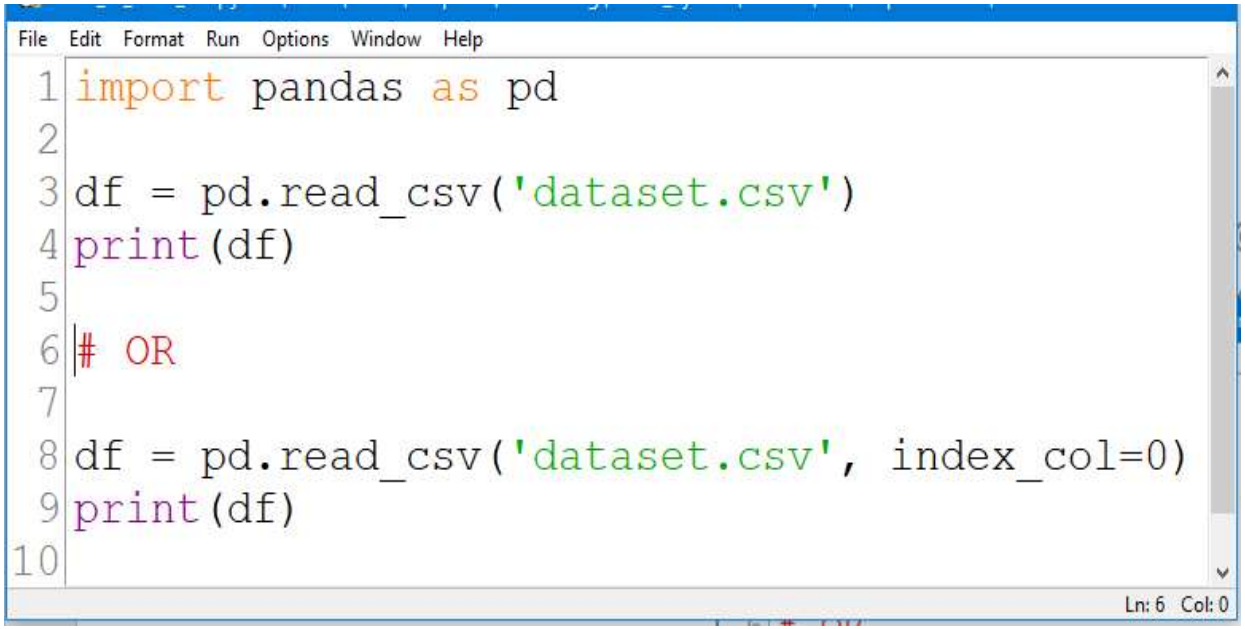
	0	1	2	3
row_1	3	2	1	0
row_2	a	b	c	d

```
data = {'row_1':[3, 2, 1, 0], 'row_2':['a','b','c','d']}
pd.DataFrame.from_dict(data,
                       orient  = 'index',
                       columns = ['A','B','C','D'])
```

	A	B	C	D
row_1	3	2	1	0
row_2	a	b	c	d

Carregando Dataframes de arquivos CSV





Carregando Dataframes de arquivos CSV

É possível carregar os dados de um arquivo CSV com apenas uma linha de código

```
df = pd.read_csv('dataset.csv')
```

	Unnamed: 0	apples	oranges
0	Ahmad	3	0
1	Ali	2	3
2	Rashed	0	7
3	Hamza	1	2

- Ao carregar os dados do CSV o Pandas não irá nomear a 1ª coluna como sendo o indice das linhas, portanto se desejar fazê-lo é preciso informar, conforme abaixo

```
df = pd.read_csv('dataset.csv', index_col=0)
```

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

- *Nota: estamos definindo o indice para ser a coluna 0*

Carregando dataset

Estamos carregando o conjunto de dados de um arquivo CSV e definindo a coluna **title** para ser o indice das linhas

```
movies_df = pd.read_csv("movies.csv", index_col="title")
```

Visualizando os dados

É possível visualizer as primeiras linhas para se ter uma ideia de como está o conjunto de dados

```
movies_df.head()
```

.head() mostra por padrão as 5 primeiras linhas do seu Dataframe, porém é possivel passer o número de linhas a ser mostrado: `movies_df.head(10)` neste caso serão mostrados as 10 primeiras linhas

Para ver as últimas linhas utilize `.tail()` que também aceita um número, no exemplo abaixo será mostrada as 2 ultimas linhas:

```
movies_df.tail(2)
```

Pegando dados do Dataframe

`.info()` deve ser um dos primeiros comandos que você executa depois de carregar seus dados

`.info()` fornece os detalhes essenciais sobre seu conjunto de dados, como o número de linhas e colunas, o número de valores não nulos, que tipo de dados estão em cada coluna e quanta memória seu DataFrame está usando.

```
movies_df.info()
```

OUT:

```
<class 'pandas.core.frame.DataFrame'>
Index: 1000 entries, Guardians of the Galaxy to Nine Lives
Data columns (total 11 columns):
Rank                  1000 non-null int64
Genre                 1000 non-null object
Description           1000 non-null object
Director              1000 non-null object
Actors                1000 non-null object
Year                  1000 non-null int64
Runtime (Minutes)     1000 non-null int64
Rating                1000 non-null float64
Votes                 1000 non-null int64
Revenue (Millions)    872 non-null float64
Metascore             936 non-null float64
dtypes: float64(3), int64(4), object(4)
memory usage: 93.8+ KB
```

```
movies_df.shape
```

OUT:

```
(1000, 11)
```

Gerenciando dados duplicados

Esse conjunto de dados não tem linhas duplicadas, mas é sempre importante verificar se você não está agregando linhas duplicadas.

Para demonstrar, vamos simplesmente dobrar nossos filmes DataFrame anexando-o a si mesmo:

Usando **`append()`** retornará uma cópia sem afetar o DataFrame original. Estamos capturando essa cópia em temporário, então não estamos trabalhando com os dados reais.

Observe que **`.shape`** prova rapidamente que nossas linhas DataFrame dobraram.

```
temp_df = movies_df.append(movies_df)
temp_df.shape
```

OUT:

```
(2000, 11)
```

Agora podemos tentar descartar duplicatas:

```
temp_df = temp_df.drop_duplicates()
temp_df.shape
```

OUT:

```
(1000, 11)
```

Gerenciando dados duplicados

Como o **`append()`**, o **`drop_duplicates()`** método também retornará uma cópia do seu DataFrame, mas desta vez com duplicatas removidas. Invocando **`.shape`** confirma que estamos de volta às 1000 linhas do nosso conjunto de dados original.

É um pouco detalhado continuar atribuindo DataFrames à mesma variável, como neste exemplo. Por esta razão, pandas tem o argumento o parâmetro **inplace** em muitos de seus métodos. Usando **`inplace=True`** modificará o objeto DataFrame no lugar:

```
temp_df.drop_duplicates(inplace=True)
```

Outro argumento importante para **`drop_duplicates()`** é manter, que tem três opções possíveis:

- **first**: (default) Descarta duplicatas, exceto para a primeira ocorrência.
- **last**: Descarta duplicatas, exceto para a última ocorrência.
- **False**: Descarta todas as duplicatas.

Compreendendo as variaveis

Usando **.describe()** em um DataFrame inteiro podemos obter um resumo da distribuição de variáveis contínuas:

```
movies_df.describe()
```

OUT:

	rank	year	runtime	rating	
count	1000.000000	1000.000000	1000.000000	1000.000000	1.0
mean	500.500000	2012.783000	113.172000	6.723200	1.69
std	288.819436	3.205962	18.810908	0.945429	1.88
min	1.000000	2006.000000	66.000000	1.900000	6.10
25%	250.750000	2010.000000	100.000000	6.200000	3.6
50%	500.500000	2014.000000	111.000000	6.800000	1.10
75%	750.250000	2016.000000	123.000000	7.400000	2.3
max	1000.000000	2016.000000	191.000000	9.000000	1.79

.describe() também pode ser usado em uma variável categórica para obter a contagem de linhas, contagem exclusiva de categorias, categoria principal e freq da categoria principal:

```
movies_df['genre'].describe()
```

OUT:

```
count                       1000
unique                       207
top       Action,Adventure,Sci-Fi
freq                          50
Name: genre, dtype: object
```

Isso nos diz que a coluna de gênero tem 207 valores únicos, o valor superior é Ação/Aventura/Sci-Fi, que aparece 50 vezes (freq).

Mais exemplos

```
import pandas as pd
data = [1,2,3,10,20,30]
df = pd.DataFrame(data)
print(df)
```

```
    0
0   1
1   2
2   3
3  10
4  20
5  30
```

```
import pandas as pd
data = {'Name' : ['AA', 'BB'], 'Age': [30,45]}
df = pd.DataFrame(data)
print(df)
```

```
   Name  Age
0    AA   30
1    BB   45
```

Mais exemplos

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data)
print(df)
```

```
   a   b     c
0  1   2   NaN
1  5  10  20.0
```

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data, index=['first', 'second'])
print(df)
```

```
        a   b     c
first   1   2   NaN
second  5  10  20.0
```

Mais exemplos

E.g. Isso mostra como criar um DataFrame com uma lista de dicionários, índices de linha e índices de coluna.

```
import pandas as pd
data = [{'a': 1, 'b': 2},{'a': 5, 'b': 10, 'c': 20}]

#With two column indices, values same as dictionary keys
df1 = pd.DataFrame(data,index=['first','second'],columns=['a','b'])

#With two column indices with one index with other name
df2 = pd.DataFrame(data,index=['first','second'],columns=['a','b1'])

print(df1)
print('...........')
print(df2)
```

```
        a   b
first   1   2
second  5  10
...........
        a  b1
first   1 NaN
second  5 NaN
```

Mais exemplos:
Criar um DataFrame a partir do Dict of Series

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3]   , index=['a', 'b', 'c']),
     'two' : pd.Series([1,2, 3, 4], index=['a', 'b', 'c', 'd'])
    }
df = pd.DataFrame(d)
print(df)
```

	one	two
a	1.0	1
b	2.0	2
c	3.0	3
d	NaN	4

Mais exemplos: Adição de coluna

```
import pandas as pd
d = {'one':pd.Series([1,2,3],   index=['a','b','c']),
     'two':pd.Series([1,2,3,4], index=['a','b','c','d'])
    }
df = pd.DataFrame(d)
# Adding a new column to an existing DataFrame object
# with column label by passing new series

print("Adding a new column by passing as Series:")
df['three'] = pd.Series([10,20,30],index=['a','b','c'])
print(df)

print("Adding a column using an existing columns in
DataFrame:")
df['four'] = df['one']+df['three']
print(df)
```

```
Adicionando uma coluna usando Série:
   one  two  three
a  1.0    1   10.0
b  2.0    2   20.0
c  3.0    3   30.0
d  NaN    4    NaN

Adicionando uma coluna usando colunas:
   one  two  three  four
a  1.0    1   10.0  11.0
b  2.0    2   20.0  22.0
c  3.0    3   30.0  33.0
d  NaN    4    NaN   NaN
```

Mais exemplos: Exclusão de coluna

```
# Using the previous DataFrame, we will delete a column
# using del function
import pandas as pd
d = {'one'   : pd.Series([1, 2, 3],    index=['a', 'b', 'c']),
     'two'   : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd']),
     'three' : pd.Series([10,20,30],   index=['a','b','c'])
    }
df = pd.DataFrame(d)
print ("Our dataframe is:")
print(df)

# using del function
print("Deleting the first column using DEL function:")
del df['one']
print(df)

# using pop function
print("Deleting another column using POP function:")
df.pop('two')
print(df)
```

```
Nosso dataframe é:
   one  two  three
a  1.0    1   10.0
b  2.0    2   20.0
c  3.0    3   30.0
d  NaN    4    NaN

Excluindo a primeira coluna:
    two  three
a    1   10.0
b    2   20.0
c    3   30.0
d    4    NaN

Excluindo outra coluna:
a   10.0
b   20.0
c   30.0
d    NaN
```

Mais exemplos: fatiamento em DataFrames

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3],    index=['a', 'b', 'c']),
     'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c','d'])
    }
df = pd.DataFrame(d)
print(df[2:4])
```

```
   one    two
c  3.0    3
d  NaN    4
```

Mais exemplos: adição de linhas

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3],    index=['a', 'b', 'c']),
     'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c','d'])
    }
df = pd.DataFrame(d)
print(df)

df2 = pd.DataFrame([[5,6], [7,8]], columns = ['a', 'b'])
df = df.append(df2 )
print(df)
```

```
   one   two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4


   one  two    a    b
a  1.0  1.0  NaN  NaN
b  2.0  2.0  NaN  NaN
c  3.0  3.0  NaN  NaN
d  NaN  4.0  NaN  NaN
0  NaN  NaN  5.0  6.0
1  NaN  NaN  7.0  8.0
```

Mais exemplos: Exclusão de linhas

```
import pandas as pd
d = {'one':pd.Series([1, 2, 3],    index=['a','b','c']),
     'two':pd.Series([1, 2, 3, 4], index=['a','b','c','d'])
    }
df = pd.DataFrame(d)
print(df)

df2 = pd.DataFrame([[5,6], [7,8]], columns = ['a', 'b'])
df = df.append(df2 )
print(df)

df = df.drop(0)
print(df)
```

```
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4

   one  two    a    b
a  1.0  1.0  NaN  NaN
b  2.0  2.0  NaN  NaN
c  3.0  3.0  NaN  NaN
d  NaN  4.0  NaN  NaN
0  NaN  NaN  5.0  6.0
1  NaN  NaN  7.0  8.0

   one  two    a    b
a  1.0  1.0  NaN  NaN
b  2.0  2.0  NaN  NaN
c  3.0  3.0  NaN  NaN
d  NaN  4.0  NaN  NaN
1  NaN  NaN  7.0  8.0
```

Mais exemplos: Reindexação

```
import pandas as pd
# Creating the first dataframe
df1 = pd.DataFrame({"A":[1, 5, 3, 4, 2],
                    "B":[3, 2, 4, 3, 4],
                    "C":[2, 2, 7, 3, 4],
                    "D":[4, 3, 6, 12, 7]},
                    index =["A1", "A2", "A3", "A4", "A5"])

# Creating the second dataframe
df2 = pd.DataFrame({"A":[10, 11, 7, 8, 5],
                    "B":[21, 5, 32, 4, 6],
                    "C":[11, 21, 23, 7, 9],
                    "D":[1, 5, 3, 8, 6]},
                    index =["A1", "A3", "A4", "A7", "A8"])

# Print the first dataframe
print(df1)
print(df2)
# find matching indexes
df1.reindex_like(df2)
```

- Pandas **`dataframe.reindex_like()`** Função retornar um objeto com índices correspondentes para mim mesmo.
- Quaisquer índices não correspondentes são preenchidos com valores de NaN.

Out[72]:

	A	B	C	D
A1	1.0	3.0	2.0	4.0
A3	3.0	4.0	7.0	6.0
A4	4.0	3.0	3.0	12.0
A7	NaN	NaN	NaN	NaN
A8	NaN	NaN	NaN	NaN

Mais exemplos:
Concatenando objetos (quadros de dados)

```
import pandas as pd
df1 = pd.DataFrame({'Name':['A','B'], 'SSN':[10,20], 'marks':[90, 95] })
df2 = pd.DataFrame({'Name':['B','C'], 'SSN':[25,30], 'marks':[80, 97] })
df3 = pd.concat([df1, df2])
df3
```

Manipulação de dados categóricos

Há muitos dados que são repetitivos por exemplo gênero, país, e os códigos são sempre repetitivos.

As variáveis categóricas podem assumir apenas um dos elementos possíveis

O tipo de dados categóricos é útil nos seguintes casos:

- Uma variável de cadeia de caracteres que consiste em apenas alguns valores diferentes. É possível converter essa variável de cadeia de caracteres em uma variável categórica, isto economizará um pouco de memória.
- A ordem lexical de uma variável não é a mesma que a ordem lógica ("um", "dois", "três").
- Ao converter para uma ordem categórica e especificar uma ordem nas categorias, a classificação e min/max usarão a ordem lógica em vez da ordem lexical. Ou seja será possível comparar os valores categóricos entre si, sem seguir a ordem lexical que é estabelecida pelas letras da palavra.
- Como um sinal para outras bibliotecas python de que esta coluna deve ser tratada como uma variável categórica (por exemplo, para usar métodos estatísticos adequados ou tipos de plotagem).

Exemplos

```
import pandas as pd
cat = pd.Categorical(['a', 'b', 'c', 'a', 'b', 'c'])
print(cat)
```

```
import pandas as pd
import numpy as np
cat = pd.Categorical(["a", "c", "c", np.nan], categories=["b", "a", "c"])
df = pd.DataFrame({"cat": cat, "s":["a", "c", "c", np.nan]})
print(df.describe())
print(df["cat"].describe())
```

Filtrando

Para subdefinir os dados, podemos aplicar a indexação booleana. Essa indexação é comumente conhecida como filtro.

Qualquer operador booleano pode ser usado para subdefinir os dados:

> maior	>= maior ou igual
< menor	<= menor ou igual
== igual	!= não é igual

Filtrando

Por exemplo, com base no DataFrame de Salarios que pode ser obtido em https://www.kaggle.com/code/vahidehdashti/first-code-sf-salaries-with-pandas-practice/input contendo diversos nomes e salários

Id	Employee...	JobTitle	BasePay	OvertimeP...	OtherPay
1	NATHANIEL FORD	GENERAL MANAGER-METROPOLITAN TRANSIT AUTHORITY	167411.18	0.0	400184.25
2	GARY JIMENEZ	CAPTAIN III (POLICE DEPARTMENT)	155966.02	245131.88	137811.38
3	ALBERT PARDINI	CAPTAIN III (POLICE DEPARTMENT)	212739.13	106088.18	16452.6
4	CHRISTOPHER CHONG	WIRE ROPE CABLE MAINTENANCE MECHANIC	77916.0	56120.71	198306.9
5	PATRICK GARDNER	DEPUTY CHIEF OF DEPARTMENT, (FIRE DEPARTMENT)	134401.6	9737.0	182234.59
6	DAVID SULLIVAN	ASSISTANT DEPUTY CHIEF II	118602.0	8601.0	189082.74
7	ALSON LEE	BATTALION CHIEF, (FIRE DEPARTMENT)	92492.01	89062.9	134426.14

Se quisermos subdefinir as linhas em que o valor do salário é maior que $120K:

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

salarios_sub = salarios[ salarios["BasePay"] > 12000 ]
```

Agrupando

Usando o método "group by" podemos:

- Dividir os dados em grupos com base em alguns critérios
- Calcular estatísticas (ou aplicar uma função) a cada grupo

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

cargos = salarios.groupby(["JobTitle"])
cargos.mean("TotalPay")
```

JobTitle	Id	TotalPay	TotalPayBenefits	Year	Notes
ACCOUNT CLERK	25734.819277	44035.664337	44035.664337	2011.00	NaN
ACCOUNTANT	24159.200000	47429.268000	47429.268000	2011.00	NaN
ACCOUNTANT INTERN	28128.833333	29031.742917	29031.742917	2011.00	NaN
ACPO,JuvP, Juv Prob (SFERS)	135284.000000	62290.780000	80266.370000	2014.00	NaN
ACUPUNCTURIST	18379.000000	67594.400000	67594.400000	2011.00	NaN

Agrupando

Uma vez que o objeto **groupby** é criado, podemos calcular várias estatísticas para cada grupo:

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

cargos = salarios.groupby(["JobTitle"])
cargos[["TotalPay"]].mean()
```

Nota: Se colchetes simples são usados para especificar a coluna (por exemplo, salário), então a saída é o objeto Pandas Series. Quando colchetes duplos são usados, a saída é um DataFrame

JobTitle	TotalPay
ACCOUNT CLERK	44035.664337
ACCOUNTANT	47429.268000
ACCOUNTANT INTERN	29031.742917
ACPO,JuvP, Juv Prob (SFERS)	62290.780000
ACUPUNCTURIST	67594.400000

Agrupando

Notas de desempenho do GroupBy:

- Nenhum agrupamento/divisão ocorre até que seja necessário. A criação do objeto groupby apenas verifica se você passou por um mapeamento válido
- Por padrão, as chaves de grupo são classificadas durante a operação GroupBy. Você pode querer passar sort=False para acelerar potencialmente:

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

cargos = salarios.groupby(["JobTitle"], sort=False)
cargos[["TotalPay"]].mean()
```

Fatiando

- Há várias maneiras de subdefinir o Quadro de Dados:
  - uma ou mais colunas
  - uma ou mais linhas
  - um subconjunto de linhas e colunas
- Linhas e colunas podem ser selecionadas por sua posição ou rótulo

Fatiando

- Ao selecionar uma coluna, é possível usar um único conjunto de colchetes, mas o objeto resultante será uma Série (não um DataFrame):

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

cargos = salarios["JobTitle"]
```

Fatiando

- Quando precisamos selecionar mais de uma coluna e/ou fazer com que a saída seja um DataFrame, devemos usar colchetes duplos:

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

cargos = salarios[["JobTitle", "TotalPay"]]
```

Selecionando Linhas

- Se precisarmos selecionar um intervalo de linhas, podemos especificar o intervalo usando ":"

```
import pandas as pd
salarios = pd.read_csv("Salaries.csv")

primeiros_10 = salarios[10:20]
```

- Observe que a primeira linha tem uma posição 0 e o último valor no intervalo é omitido:
- Assim, para o intervalo 0:10, as primeiras 10 linhas são retornadas com as posições começando com 0 e terminando com 9

References

Al-Sharif, Ziad; Introduction To Pandas DataFrame

Pandas: documentation
- https://pandas.pydata.org/pandas-docs/stable/index.html

Pandas: Input/output
- https://pandas.pydata.org/pandas-docs/stable/reference/io.html

Pandas: DataFrame
- https://pandas.pydata.org/pandas-docs/stable/reference/frame.html

Pandas: Series
- https://pandas.pydata.org/pandas-docs/stable/reference/series.html

Pandas: Plotting
- https://pandas.pydata.org/pandas-docs/stable/reference/plotting.html